LABORATORY WORK NO. 1

The Caesar Cipher

Implementation for the Romanian alphabet, with a keyword-based permutation of the alphabet

---

Discipline: **Cryptography**

Author: **Mutu Adrian**

Group: **FAF-242**

9 October 2026

1. Short theory

The Caesar cipher is a monoalphabetic substitution in which every letter of the plaintext is replaced by the letter found a fixed number of positions further along the alphabet. The secret key `k` is that shift, and the same key serves both directions:

$$c = e_k(x) = (x + k) \bmod n$$

$$m = d_k(y) = (y - k) \bmod n$$

Here `x` and `y` are the numeric codes of a letter — its position in the alphabet, counted from 0 — and `n` is the length of the alphabet. When `y − k` is negative the result is brought back into the range `0 … n−1` by adding `n`, because some languages return negative remainders.

For the Romanian alphabet `n = 31`, so `k ∈ {1, 2, …, 30}`. The value `k = 0` is excluded because it would leave the text unchanged. A key space of 30 is the cipher's fatal weakness: an attacker decrypts the message with all 30 keys and reads the one candidate that makes sense in the language of the message.

Adding a permutation

To enlarge the key space, the alphabet itself is reordered using a second key — a keyword. The permuted alphabet is built by writing the letters of the keyword first, keeping only the first occurrence of a repeated letter, followed by the remaining letters of the alphabet in their natural order. The shift is then applied over this new order, so a letter's numeric code becomes its position in the *permuted* alphabet rather than in the natural one.

The two keys `(k₁, k₂)` give a far larger key space:

Version	Keys	Number of keys	Broken by
Caesar	`k`	30	exhaustive search — 30 trial decryptions
Caesar with a permutation	`(k₁, k₂)`	$31! \times 30 \approx 2.47 \times 10^{35}$	frequency analysis

Exhaustive search is now out of reach. The cipher is still broken easily, though, because it remains a monoalphabetic substitution: each plaintext letter is always replaced by the same ciphertext letter, so the letter frequencies of the language pass through into the ciphertext unchanged. Different key pairs can also produce the same substitution, so the number of distinct encryptions never exceeds 31!.

2. Implementation

The program is written in Python 3 and uses only the standard library. Both tasks share one implementation of the cipher, because they differ in nothing but the order of the alphabet the shift is applied to.

File	Contents
`romanian_alphabet.py`	the letter encoding of Table 2 and all input validation
`caesar.py`	the permuted alphabet, encryption and decryption
`main.py`	the interactive program for both tasks
`tests/test_ciphers.py`	automated checks, including the examples worked out by hand in the assignment

The letter encoding

```
ALPHABET = (
    "A", "Ă", "Â", "B", "C", "D", "E", "F", "G", "H", "I", "Î", "J", "K", "L",
    "M", "N", "O", "P", "Q", "R", "S", "Ș", "T", "Ț", "U", "V", "W", "X", "Y",
    "Z",
)

N = len(ALPHABET)  # 31

LETTER_TO_CODE = {letter: code for code, letter in enumerate(ALPHABET)}
```

The alphabet is an explicit tuple in the order of Table 2, so a letter's numeric code is simply its index in that tuple: `A = 0`, `Ă = 1`, …, `Z = 30`. This is what satisfies the requirement that built-in character encodings must not be used to compute the shift — no `ord()` or `chr()` appears anywhere in the ciphers.

Building the permuted alphabet

```
def build_permuted_alphabet(keyword, alphabet=ALPHABET):
    permuted = []
    for letter in keyword:
        if letter not in permuted:
            permuted.append(letter)
    for letter in alphabet:
        if letter not in permuted:
            permuted.append(letter)
    return tuple(permuted)
```

The first loop writes the letters of the keyword, skipping any letter already present, so only the first occurrence of a repeated letter is kept. The second loop appends the letters not used by the keyword in their natural order. The result is always a permutation of all 31 letters.

Encryption and decryption

```
def encrypt(text, key, alphabet=ALPHABET):
    to_code = _codes(alphabet)
    n = len(alphabet)
    return "".join(alphabet[(to_code[letter] + key) % n] for letter in text)


def decrypt(text, key, alphabet=ALPHABET):
    to_code = _codes(alphabet)
    n = len(alphabet)
    return "".join(alphabet[(to_code[letter] - key + n) % n] for letter in text)
```

These are the two formulas applied letter by letter: each letter is turned into its code, shifted modulo `n`, and turned back into the letter standing at the new position. The ordered alphabet is a parameter, so task 1.1 calls these functions with `ALPHABET` and task 1.2 calls them with the permuted alphabet — one implementation for both tasks. In `decrypt`, `+ n` before the modulo keeps the result in `0 … n−1` even in a language whose remainder may come out negative.

Input validation

```
def normalize_text(raw):
    text = unicodedata.normalize("NFC", raw)

    letters = []
    for position, char in enumerate(text, start=1):
        if char.isspace():          # spaces are allowed in the input...
            continue                # ...and simply dropped
        char = CEDILLA_VARIANTS.get(char, char)
        char = LOWER_TO_UPPER.get(char, char)
        if char not in LETTER_TO_CODE:
            raise InvalidCharacterError(char, position)
        letters.append(char)
    return "".join(letters)
```

The text is converted to uppercase, spaces are removed, and the cedilla spellings `Ş` / `Ţ` are folded onto the comma-below letters `Ș` / `Ț` so that text typed on different keyboards is accepted. NFC normalization first recomposes letters that arrive as a base letter plus a combining comma. The first character that is not a Romanian letter raises an error carrying both the character and its position, which is what lets the program tell the user exactly what was rejected.

```
def parse_key(raw):
    try:
        key = int(raw.strip())
    except ValueError:
        raise InvalidKeyError("... must be a whole number between 1 and 30 ...")
    if not MIN_KEY <= key <= MAX_KEY:
        raise InvalidKeyError("the key %d is out of range; ..." % key)
    return key
```

Key 1 is accepted only as an integer in `1 … 30`; key 2 is put through `normalize_text` and then checked to be at least 7 letters long. Every rejection is reported with the allowed values and the program asks for the value again:

```
def ask(prompt, parse):
    """Ask for a value until `parse` accepts it, reporting every rejection."""
    while True:
        raw = input(prompt)
        try:
            return parse(raw)
        except (InvalidKeyError, InvalidCharacterError, InvalidKeywordError) as error:
            print("  [invalid input] %s\n" % error)
```

This single helper drives every prompt in the program, so no invalid value can reach the cipher and no invalid value ends the program.

3. Results

All runs below use the plaintext `cifrul cezar` and the numeric key `k = 3`, so the two tasks can be compared directly. The console output is reproduced as it appears on screen.

Task 1.1 — encryption

```
======================================================================
Task 1.1 - Caesar cipher (Romanian alphabet, n = 31)
======================================================================
Operation - 1 = encrypt, 2 = decrypt: 1
Key 1 - numeric shift (1..30): 3

  Alphabet in use (Table 2):
    code    0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
    letter  A  Ă  Â  B  C  D  E  F  G  H  I  Î  J  K  L  M
    code   16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
    letter  N  O  P  Q  R  S  Ș  T  Ț  U  V  W  X  Y  Z

Plaintext (spaces are allowed and removed): cifrul cezar

  Plaintext  (uppercase, no spaces) : CIFRULCEZAR
  Ciphertext                        : FKITXOFHÂBT
```

The result can be checked by hand: `C = 4` becomes `4 + 3 = 7 = F`, `I = 10` becomes `13 = K`, and `Z = 30` wraps around to `(30 + 3) mod 31 = 2 = Â`.

Task 1.1 — decryption

```
Operation - 1 = encrypt, 2 = decrypt: 2
Key 1 - numeric shift (1..30): 3

Ciphertext (spaces are allowed and removed): FKITXOFHÂBT

  Ciphertext (uppercase, no spaces) : FKITXOFHÂBT
  Decrypted message                 : CIFRULCEZAR
```

Task 1.2 — encryption with the keyword `criptografie`

```
======================================================================
Task 1.2 - Caesar cipher with a keyword permutation (Romanian alphabet, n = 31)
======================================================================
Operation - 1 = encrypt, 2 = decrypt: 1
Key 1 - numeric shift (1..30): 3
Key 2 - keyword (at least 7 Romanian letters): criptografie

  Keyword used: CRIPTOGRAFIE
  Permuted alphabet (check it by hand against Table 3):
    code    0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
    letter  C  R  I  P  T  O  G  A  F  E  Ă  Â  B  D  H  Î
    code   16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
    letter  J  K  L  M  N  Q  S  Ș  Ț  U  V  W  X  Y  Z

Plaintext (spaces are allowed and removed): cifrul cezar

  Plaintext  (uppercase, no spaces) : CIFRULCEZAR
  Ciphertext                        : POÂTXQPBIĂT
```

The keyword `CRIPTOGRAFIE` contributes the 10 distinct letters C, R, I, P, T, O, G, A, F, E — the second R, I and the repeated letters are dropped — and the remaining 21 letters of the alphabet follow in their natural order. In this permuted alphabet `C` has position 0 and becomes the letter at position 3, which is `P`; `F` has position 8 and becomes `Â`.

The same plaintext and the same numeric key now give a different ciphertext:

Version	Ciphertext of `CIFRULCEZAR`, `k = 3`
Task 1.1 — plain Caesar	`FKITXOFHÂBT`
Task 1.2 — with the keyword `CRIPTOGRAFIE`	`POÂTXQPBIĂT`

Task 1.2 — decryption

```
Operation – 1 = encrypt, 2 = decrypt: 2
Key 1 – numeric shift (1..30): 3
Key 2 – keyword (at least 7 Romanian letters): criptografie

  Keyword used: CRIPTOGRAFIE
  Permuted alphabet (check it by hand against Table 3):
    code    0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
    letter  C  R  I  P  T  O  G  A  F  E  Ă  Â  B  D  H  Î
    code   16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
    letter  J  K  L  M  N  Q  S  Ș  Ț  U  V  W  X  Y  Z

Ciphertext (spaces are allowed and removed): POÂTXQPBIĂT

  Ciphertext (uppercase, no spaces) : POÂTXQPBIĂT
  Decrypted message                 : CIFRULCEZAR
```

Invalid input — the numeric key

A key of 0, a key above 30 and a key that is not a number are each rejected with the allowed range, and the program asks again:

```
Operation – 1 = encrypt, 2 = decrypt: 1
Key 1 – numeric shift (1..30): 0
  [invalid input] the key 0 is out of range; key 1 must be between 1 and 30 inclusive (the
alphabet has 31 letters, and a shift of 0 would leave the text unchanged)

Key 1 – numeric shift (1..30): 35
  [invalid input] the key 35 is out of range; key 1 must be between 1 and 30 inclusive (the
alphabet has 31 letters, and a shift of 0 would leave the text unchanged)

Key 1 – numeric shift (1..30): trei
  [invalid input] 'trei' is not an integer; key 1 must be a whole number between 1 and 30
inclusive

Key 1 – numeric shift (1..30): 3
```

Invalid input — a character outside the alphabet

The message names the rejected character and its position, then lists the letters that are allowed:

```
Plaintext (spaces are allowed and removed): salut, lume
  [invalid input] the character ',' (position 6) is not a letter of the Romanian alphabet;
allowed letters are A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z (uppercase or
lowercase) and spaces, which are removed

Plaintext (spaces are allowed and removed): salut lume

  Plaintext  (uppercase, no spaces) : SALUTLUME
  Ciphertext                        : ȚBOXVOXPH
```

Invalid input — the keyword

A keyword shorter than 7 letters and a keyword containing a character outside the alphabet are both rejected:

```
Key 2 — keyword (at least 7 Romanian letters): cripto
  [invalid input] the keyword 'CRIPTO' has 6 letter(s); key 2 must have at least 7 letters

Key 2 — keyword (at least 7 Romanian letters): cripto-grafie
  [invalid input] key 2 may contain only letters of the Romanian alphabet: the character '-'
(position 7) is not a letter of the Romanian alphabet; allowed letters are A Ă Â B C D E F G H I
Î J K L M N O P Q R S Ș T Ț U V W X Y Z (uppercase or lowercase) and spaces, which are removed

Key 2 — keyword (at least 7 Romanian letters): criptografie
```

Automated checks

The repository also contains a test suite that pins the implementation to the examples worked out by hand in the assignment. Because `encrypt` and `decrypt` take the ordered alphabet as a parameter, the very same functions can be run over the 26-letter English alphabet and compared with the values printed in the laboratory work:

Input	Keys	Expected	Alphabet
`BRUTEFORCEATTACK`	`k = 17`	`SILKVWFITVRKKRTB`	English, n = 26
`CIFRULCEZAR`	`k = 3`	`FLIUXOFHCDU`	English, n = 26
`CIFRULCEZAR`	`k = 3`, k_2 = `CRYPTOGRAPHY`	`PLKTXQPJYDT`	English, permuted
`CIFRULCEZAR`	`k = 3`	`FKITXOFHÂBT`	Romanian, n = 31

All four match, and a round-trip test confirms that `decrypt(encrypt(m, k), k) = m` for every key from 1 to 30 in both tasks:

```
$ python3 -m unittest discover -s tests
....................
----------------------------------------------------------------------
Ran 20 tests in 0.001s

OK
```

4. Conclusions

Neither cipher is usable for real protection, but they fail for different reasons, and that difference is the point of the laboratory work.

The plain Caesar cipher is broken by its key space alone. With `n = 31` there are only 30 possible keys, so an attacker writes out all 30 decryptions and picks the readable one — a few seconds of work, and no knowledge of the plaintext is needed beyond knowing what language it is in. Secrecy of the method adds nothing, since the method is public.

Adding the keyword permutation raises the number of keys to about 2.47×10^{35}, which does put exhaustive search out of reach. It does not make the cipher strong, though, because the weakness simply moves: the cipher is still a monoalphabetic substitution, so each plaintext letter always maps to the same ciphertext letter and the letter frequencies of Romanian survive into the ciphertext. An attacker counts

letters, matches the most frequent ones against the frequencies of the language, and recovers the substitution without ever searching the key space. A large key space is therefore a necessary condition for security, not a sufficient one — the structure the cipher leaves in the ciphertext matters more than the number of keys.

The practical lesson from the implementation was that the alphabet must be treated as explicit data rather than as something the programming language already knows. Once the alphabet was a plain ordered list and a letter's code was its position in that list, the Romanian diacritics stopped being a special case: `Ă`, `Â`, `Î`, `Ș` and `Ț` are simply letters at positions 1, 2, 11, 22 and 24, and wrapping `Z` around to `Â` needs no extra handling. Making the ordered alphabet a parameter of the two cipher functions then collapsed both tasks into one implementation, which is also what made it possible to verify the code against the English examples printed in the assignment before trusting it on 31 letters. Most of the remaining work was input validation — in a program of this size, the checks on the key, the keyword and the text are longer than the cipher itself.

5. Source code

The full source code is in the repository github.com/alo42/CS.

To run the program:

```
git clone git@github.com:alo42/CS.git
cd CS
python3 main.py
```

To run the automated checks:

```
python3 -m unittest discover -s tests
```

The program needs only Python 3 and the standard library.